Deployment of DistilBERT for Sentiment Analysis in Docker and Hugging Face Spaces

Tamara Besedina

Abstract

This work deploys the pre-trained DistilBERT deep learning model for binary sentiment analysis of English-language reviews. The main local implementation is divided into two Docker containers. The first container contains the model and the FastAPI inference service, while the second contains only the Gradio graphical interface and an HTTP client. The containers are built and launched together using Docker Compose. In addition, a cloud application was created in Hugging Face Spaces using Gradio, ZeroGPU, and a published API endpoint.

The solution was verified in three ways: with a direct HTTP request to the local FastAPI through the terminal, with inference through the local graphical interface, and with a remote call through the terminal and the Hugging Face Space web interface using the Gradio API. The state of the containers and their CPU, memory, network, Input/Output, and process consumption were also analyzed.

The author also improved the Dockerfile and Compose configuration and set resource limit parameters.

The code is available on https://github.com/tomabes/distilbert-sentiment-analysis-docker.

1. Selected Model and Inference Task

A compact version of the BERT model, fine-tuned for binary sentiment analysis on SST-2 (Stanford Sentiment Treebank), was used: **distilbert-base-uncased-finetuned-sst-2-english**. The output contains:

- `label`: `POSITIVE` or `NEGATIVE`;
- `score`: the model's confidence in the predicted class.

DistilBERT was selected because it is already pre-trained and publicly available, and its result is easy to verify manually. No training was performed.

2. Overall Project Architecture

Main architecture:

```
User in a browser
        |
        | http://localhost:7860
        v
+-------------------------+
| UI container            |
| Gradio + requests       |
| distilbert-ui:latest    |
+-------------------------+
        |
        | POST http://api:8000/predict
        | JSON: {"text": "..."}
        v
+-------------------------+
| API container           |
| FastAPI + Transformers  |
| PyTorch + DistilBERT    |
| distilbert-api:latest   |
+-------------------------+
        |
        v
{"label": "...", "score": ...}
```

Cloud version:

```
User or Gradio Client
        |
        v
+---------------------------------+
| Hugging Face Space              |
| Gradio + Transformers + model   |
| @spaces.GPU -> ZeroGPU          |
+---------------------------------+
```

Hugging Face Space: https://huggingface.co/spaces/Yabusiki/distilbert-sentiment-analysis.

2.1. File Structure

```
docker-dl-assignment/
├── api/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py
│   │   ├── model.py
│   │   └── schemas.py
│   ├── .dockerignore
│   ├── Dockerfile
│   ├── download_model.py
│   └── requirements.txt
├── ui/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── client.py
│   │   └── main.py
│   ├── .dockerignore
│   ├── Dockerfile
│   └── requirements.txt
├── hf_space/
│   ├── README.md
│   ├── app.py
│   └── requirements.txt
├── scripts/
│   ├── call_hf_space.py
│   ├── run_api_container.sh
│   └── test_local_api.sh
├── report_assets/
├── .dockerignore
├── docker-compose.yml
└── README.md
```

2.2. Local Environment Setup

The project was completed on a MacBook Air with an M1 processor, 8 CPU cores, 8 GB of unified memory, and ARM64 architecture. On macOS, Linux containers are not run by the macOS kernel, so Docker Desktop was installed. In addition to the graphical interface, it provides the Docker CLI accessed by the terminal; the Docker daemon, which manages images and containers; a Linux VM in which Linux containers run; and virtual storage for images, layers, and containers.

While Docker Desktop was running, commands were entered in Terminal.

The availability of the client tools was checked with the following commands:

```
docker --version
docker compose version
```

The following versions were obtained:

```
Docker version 29.9.2
Docker Compose version v5.6.1
```

This command confirmed that the Compose file can be parsed syntactically and defines both services:

```
docker compose config --services
```

Result:

```
api
ui
```

3. `api` Inference Service

3.1. Purpose

The API container is responsible only for ML inference:

1. it accepts an HTTP request containing text;
2. validates the JSON structure;
3. loads the locally stored tokenizer and model;
4. runs the Transformers pipeline;
5. returns `label` and `score`.

3.2. Dependencies in `api/requirements.txt`

- **FastAPI (fastapi)** — a Python framework for creating an HTTP API; it describes routes and data.
- **Uvicorn (uvicorn[standard])** — an ASGI server that listens on a port and forwards requests to the FastAPI application.
- **Transformers (transformers)** — loading the tokenizer, model, and ML pipeline.
- **PyTorch (torch)** — the computation backend for the neural network.

3.3. Model Preloading

The `api/download_model.py` file is executed while the image is being built:

```
MODEL_NAME = "distilbert-base-uncased-finetuned-sst-2-english"
MODEL_DIR = os.getenv("MODEL_DIR", "/models/distilbert-sst2")

def download_model() -> None:
    os.makedirs(MODEL_DIR, exist_ok=True)

    tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
    model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME)

    tokenizer.save_pretrained(MODEL_DIR)
    model.save_pretrained(MODEL_DIR)
```

`AutoTokenizer.from_pretrained` downloads the tokenization rules and vocabulary; `AutoModelForSequenceClassification.from_pretrained` downloads the architecture and weights; `save_pretrained` saves both components inside the image.

The advantage of this solution is that the model does not need to be downloaded every time the container starts. The cost of this solution is a larger image and a longer initial build.

3.4. Model Loading and Prediction

The inference logic is defined in `api/app/model.py`:

```
@lru_cache(maxsize=1)
def load_model():
    return pipeline(
        "sentiment-analysis",
        model=MODEL_DIR,
        tokenizer=MODEL_DIR,
    )
```

The pipeline receives a local path inside the image. The `@lru_cache(maxsize=1)` decorator stores the first pipeline that is created and returns the same object. Therefore, the model is loaded into memory once.

Prediction function:

```
def predict_sentiment(text: str) -> dict[str, float | str]:
    classifier = load_model()
    result = classifier(text)[0]

    return {
        "label": result["label"],
        "score": float(result["score"]),
    }
```

In this function, `load_model()` returns the cached pipeline; the tokenizer converts the text into token IDs; PyTorch runs DistilBERT; the pipeline creates a list of results; `[0]` selects the result for the first text; and the function returns a Python dictionary.

3.5. Data Schemas

`api/app/schemas.py` contains Pydantic models:

```
class PredictionRequest(BaseModel):
    text: str = Field(min_length=1)

class PredictionResponse(BaseModel):
    label: str
    score: float
```

`PredictionRequest` requires a non-empty string field named `text`. If the JSON has an incorrect structure, FastAPI returns a validation error before calling the model. `PredictionResponse` defines the response format: a string label and a numeric score.

3.6. HTTP Routes

`api/app/main.py` creates the application:

```
app = FastAPI(title="DistilBERT Inference API")
```

The health-check route indicates that the web service is available:

```
@app.get("/health")
def health_check() -> dict[str, str]:
    return {"status": "ok"}
```

Inference route:

```
@app.post("/predict", response_model=PredictionResponse)
def predict(request: PredictionRequest) -> PredictionResponse:
    prediction = predict_sentiment(request.text)
    return PredictionResponse(**prediction)
```

FastAPI converts the input JSON into `PredictionRequest`. `request.text` is passed to the model. The expression `PredictionResponse(**prediction)` expands the dictionary as named arguments.

3.7. API Dockerfile

The Dockerfile contains the following key instructions:

- `FROM python:3.11-slim` — the base Linux image with Python 3.11. The `slim` variant reduces the base layer by excluding system components not needed for the project.
- `PYTHONDONTWRITEBYTECODE=1` — do not create `.pyc` files.
- `PYTHONUNBUFFERED=1` — output logs to Docker stream.
- `HF_HOME=/models` — use `/models` for Hugging Face data.

- `MODEL_DIR` — a single path where the script saves and the application loads the model.
- `WORKDIR /app` — subsequent relative paths are resolved from `/app`.
- `COPY requirements.txt .` and a separate `RUN pip install` — the dependency layer is cached independently of code changes.
- `RUN python download_model.py` — the model becomes part of the image.
- `COPY app ./app` is performed last among the copy operations, so changing the application code does not require reinstalling the libraries and downloading the model again.
- `EXPOSE 8000` documents the internal port.
- `HEALTHCHECK` calls `/health` every 30 seconds.
- `ENTRYPOINT` starts Uvicorn and specifies the `app` object from the `app.main` module.
- `CMD` adds the address and port. `0.0.0.0` is required so that the server accepts requests not only from within its own process.

The layers are ordered so that dependencies that change infrequently precede code that changes more frequently.

3.8. Build

The build was started with the following command:

```
docker compose build
```

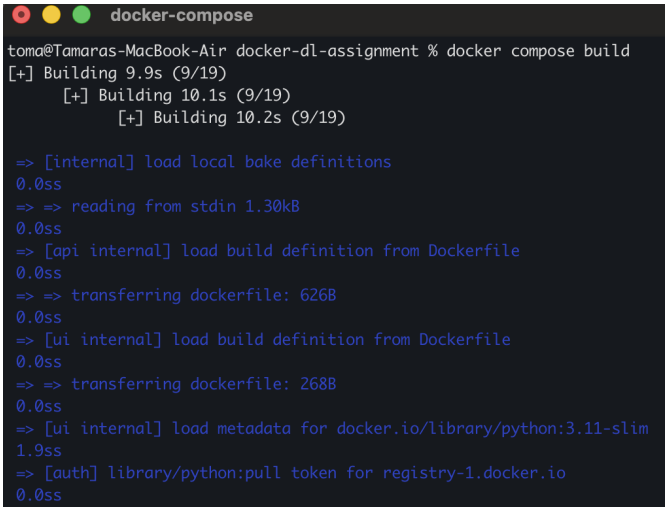


Figure 1: Docker Compose reads two Dockerfiles, obtains the python:3.11-slim base image, and starts building the API and UI services.

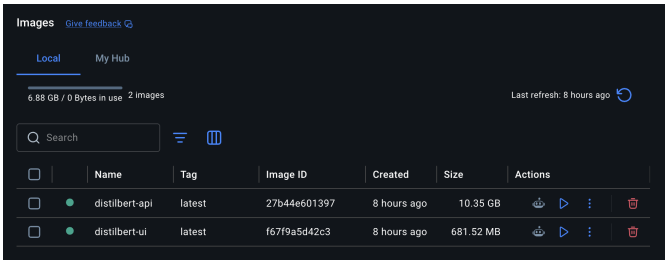


Figure 2: Two local images in Docker Desktop.

In the screenshot, Docker Desktop shows `10.35 GB` for the API and `681.52 MB` for the UI. A possible future improvement is to explicitly install a CPU-only build of PyTorch for the local API. This would reduce the image size and disk usage without slowing down local inference, because an NVIDIA GPU is not available on the Mac.

3.9. Direct Launch of the API Container

The `scripts/run_api_container.sh` script was used to create the container through the terminal:

```
docker run \
  -d \
  --rm \
  -p 8000:8000 \
  --shm-size=1g \
  --memory=4g \
  --name "$containerName" \
  distilbert-api:latest
```

Parameters:

- `-d` — run in the background;
- `--rm` — remove the container after it stops without removing the image;
- `-p 8000:8000` — port `8000` on the Mac is forwarded to port `8000` in the container;
- `--shm-size=1g` — the shared memory segment `/dev/shm` is increased to 1 GiB;
- `--memory=4g` — the memory limit;
- `--name` — a clear container name.

3.10. Direct Inference Through the API

After Compose was launched, the request was executed from the Mac terminal:

```
curl -s -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"text": "I really enjoyed this assignment."}' \
  | python3 -m json.tool
```

`curl` accesses port 8000 on the host. Docker forwards the request to the API container. The header indicates that the request body is JSON. The `-d` parameter passes the `text` field. The `python3 -m json.tool` command formats the received JSON.

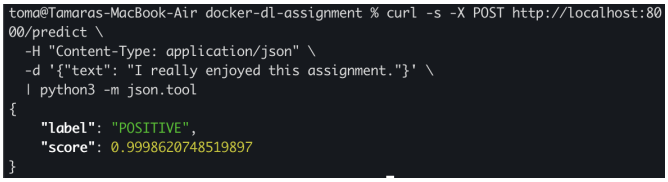


Figure 3: Successful POST request to the local FastAPI.

4. ui Graphical Service

4.1. Purpose

The UI container is responsible only for the user interface and HTTP requests to the API. It sends text over the network and receives the completed JSON response. The UI does not contain PyTorch, Transformers, DistilBERT weights, or a local ML inference function. The ML operation is performed in another container.

4.2. UI Dependencies in `ui/requirements.txt`

- **Gradio (gradio)** builds the web interface.
- **Requests (requests)** sends an HTTP POST request to FastAPI.

4.3. HTTP Client

`ui/app/client.py`:

```
API_URL = os.getenv("API_URL", "http://api:8000")

def request_prediction(text: str) -> str:
    response = requests.post(
        f"{API_URL}/predict",
        json={"text": text},
        timeout=30,
    )
    response.raise_for_status()

    data = response.json()
    return f'{data["label"]} ({data["score"]:.4f})'
```

`API_URL` is obtained from an environment variable. The default value is `http://api:8000`, where `api` is the name of the Compose service. Docker Compose creates a shared network and an embedded DNS that resolves the name `api` to the internal IP address of the API container. From the Mac, `http://localhost:8000` is used. `requests.post` serializes `json={"text": text}` and sends it to `/predict`. `timeout=30`. `raise_for_status()` converts 4xx/5xx HTTP responses into an exception. After a successful response, `response.json()` converts the JSON into a Python dictionary.

4.4. Interface Handlers

The main function in `ui/app/main.py`:

```
def classify_text(text: str) -> str:
    cleaned_text = text.strip()

    if not cleaned_text:
        return "Please enter some text."

    return request_prediction(cleaned_text)
```

`strip()` removes spaces from the beginning and end. If the string is empty after cleaning, the HTTP request is not sent. For non-empty text, the FastAPI client is called.

Two helper functions insert prepared reviews and perform analysis at the same time:

```
def use_positive_review() -> tuple[str, str]:
    return POSITIVE_REVIEW, classify_text(POSITIVE_REVIEW)

def use_negative_review() -> tuple[str, str]:
    return NEGATIVE_REVIEW, classify_text(NEGATIVE_REVIEW)
```

The tuple contains two results: the text for the input field and the prediction for the output field.

4.5. Building the Gradio UI

```
with gr.Blocks(title="DistilBERT Sentiment Analysis") as demo:
    gr.Markdown("# DistilBERT Sentiment Analysis")

    review_text = gr.Textbox(
        label="Review text",
        lines=4,
        placeholder="Type an English review here.",
    )

    with gr.Row():
        positive_button = gr.Button("Positive review")
        negative_button = gr.Button("Negative review")
        analyze_button = gr.Button("Analyze", variant="primary")

    prediction_output = gr.Textbox(label="Prediction")
```

`gr.Blocks` defines the page. `Textbox` stores the input, `Row` arranges the buttons horizontally, and the second `Textbox` displays the result.

The button event connects the components to the Python function:

```
analyze_button.click(
    fn=classify_text,
    inputs=review_text,
    outputs=prediction_output,
)
```

On click, the contents of `review_text` are passed to `classify_text`, and the returned string is placed in `prediction_output`. `review_text.submit(...)` makes it possible to run the same function by submitting the form from the keyboard.

Launch:

```
if __name__ == "__main__":
    demo.launch(server_name="0.0.0.0", server_port=7860)
```

`0.0.0.0` allows connections from outside the container. If only `127.0.0.1` were used, the server would be available only inside the UI container.

4.6. UI Dockerfile

The UI Dockerfile is simpler than the API Dockerfile because it does not download the model. The `python -m app.main` command launches `main.py`.

4.7. GUI Verification

After launch, the following address was opened:

```
http://localhost:7860
```

A negative review was entered:

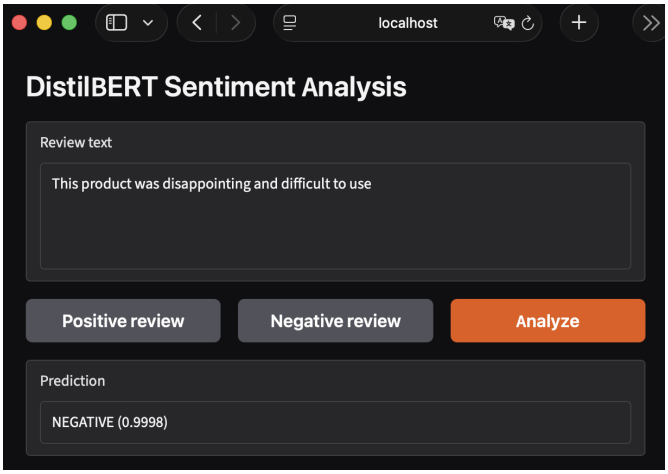


Figure 4: The local Gradio UI received the result from the separate FastAPI container.

5. Orchestration with Docker Compose

Docker Compose is part of the Docker CLI for describing and jointly launching multiple services. Its configuration is stored in the `docker-compose.yml` YAML file.

5.1. API Service

```
api:
  build:
    context: ./api
  image: distilbert-api:latest
  container_name: distilbert_api
  ports:
    - "8000:8000"
  shm_size: "1gb"
  mem_limit: "4g"
```

- `context: ./api` defines the build context. Docker sees the files inside the `api` directory.

- `image` specifies the name of the built image.
- `container_name` specifies the name of the instance.
- `ports` publishes the API on the Mac.
- `shm_size` increases shared memory.
- `mem_limit` specifies the upper RAM limit.

5.3. UI Service

```
ui:
  build:
    context: ./ui
  image: distilbert-ui:latest
  container_name: distilbert_ui
  ports:
    - "7860:7860"
  environment:
    - API_URL=http://api:8000
  depends_on:
    api:
      condition: service_healthy
  shm_size: "512mb"
  mem_limit: "1g"
```

`API_URL` passes the internal API address to the UI. `depends_on` with `condition: service_healthy` creates the API first and waits for a successful Docker health check, because starting the process does not yet guarantee that the web server is ready to accept requests.

The UI receives lower limits because it does not store the model and PyTorch.

5.4. Launch

The command created a network and two containers:

```
docker compose up
```

```
toma@Tamaras-MacBook-Air ~ % cd '/Users/toma/Downloads/STUDY & MASTERS/+UJM/M2 2
026:27/M2 1st semester/AI Infrastructure/docker-dl-assignment'
toma@Tamaras-MacBook-Air docker-dl-assignment % docker compose up
[+] up 3/3
 ✔ Network docker-dl-assignment_default Created                    0.0s
 ✔ Container distilbert_api             Created                   25.2s
 ✔ Container distilbert_ui              Created                    0.2s
Attaching to distilbert_api, distilbert_ui
Container distilbert_api Waiting
distilbert_api  | INFO:     Started server process [1]
distilbert_api  | INFO:     Waiting for application startup.
distilbert_api  | INFO:     Application startup complete.
distilbert_api  | INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C
 to quit)
distilbert_api  | INFO:     127.0.0.1:49946 - "GET /health HTTP/1.1" 200 OK
Container distilbert_api Healthy
distilbert_ui   | * Running on local URL:  http://0.0.0.0:7860
distilbert_ui   | * To create a public link, set `share=True` in `launch()`.
distilbert_api  | INFO:     127.0.0.1:55416 - "GET /health HTTP/1.1" 200 OK
```

Figure 6: The API starts Uvicorn, the health check returns `200 OK`, and the UI is then allowed to start.

5.5. Status Verification

The command showed the state (Figure 5) of the containers:

```
docker compose ps
```

5.6. Resource Consumption

A one-time snapshot was obtained with:

```
docker stats --no-stream
```

The results are presented in Table 1.

6. Hugging Face Spaces

6.1. Purpose

The additional part demonstrates that the model can be deployed outside the local computer and called through a public online API. The `Yabusiki/distilbert-sentiment-analysis` Space was created with the Gradio SDK, ZeroGPU hardware, and public visibility.

6.2. Difference in the Cloud Architecture

The local part must have two containers, so the UI calls FastAPI. In the Space, the interface and model are placed in one Python application:

```
Gradio event -> Python function -> Transformers pipeline -> result
```

6.3. Dependencies in `hf_space/requirements.txt`

Only `transformers` is installed because Gradio is already defined by the SDK configuration, while the infrastructure is provided by ZeroGPU.

6.4. Pipeline Loading

```
classifier = pipeline(
    "sentiment-analysis",
    model=MODEL_NAME,
    tokenizer=MODEL_NAME,
    device=0,
)
```

Here, the model is specified by its Hub name rather than by a local path. When the Space started, the following files were downloaded:

- `config.json`;
- `model.safetensors`, approximately 268 MB in size;
- `tokenizer_config.json`;
- `vocab.txt`.

`device=0` refers to the first GPU. The pipeline is created once at startup.

6.5. Inference and ZeroGPU

This function runs the model and formats the result:

```
def predict_sentiment(text: str) -> str:
    result = classifier(text)[0]
    return f'{result["label"]} ({float(result["score"]):.4f})'
```

Main handler:

```
@spaces.GPU(duration=30)
def classify_text(text: str) -> str:
    cleaned_text = text.strip()

    if not cleaned_text:
        return "Please enter some text."

    return predict_sentiment(cleaned_text)
```

`@spaces.GPU(duration=30)` informs ZeroGPU that the function needs a temporary GPU. `duration=30` is the requested upper GPU-time interval.

```
toma@Tamaras-MacBook-Air ~ % cd '/Users/toma/Downloads/STUDY & MASTERS/+UJM/M2 2026:27/M2 1st semester/AI Infrastructure/docker-dl-assignment'
toma@Tamaras-MacBook-Air docker-dl-assignment % docker compose ps
NAME             IMAGE                    COMMAND                  SERVICE   CREATED          STATUS                    PORTS
distilbert_api   distilbert-api:latest    "uvicorn app.main:ap…"   api       29 minutes ago   Up 29 minutes (healthy)   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
distilbert_ui    distilbert-ui:latest     "python -m app.main"     ui        29 minutes ago   Up 28 minutes             0.0.0.0:7860->7860/tcp, [::]:7860->7860/tcp
toma@Tamaras-MacBook-Air docker-dl-assignment %
```

Figure 5: Both containers are in the `Up` state; the API additionally has the `healthy` status; ports 8000 and 7860 are published.

Table 1: Snapshot of CPU, memory, network and block I/O, and the number of processes for the two containers.

Metric	UI	API	Interpretation
CPU	7.38%	0.25%	Instantaneous snapshot: the UI may have been serving the browser, while the API had already completed inference
Memory usage	126.7 MiB	991.2 MiB	The API holds PyTorch, Transformers, the tokenizer, and model weights
Memory limit	1 GiB	3.825 GiB	The UI received its limit; the API is limited by the memory actually available to the Docker VM
Memory %	12.37%	25.31%	The percentage is calculated relative to the limit shown for the container
Network I/O	83.5 kB / 428 kB	8.2 kB / 4.77 kB	The UI serves browser resources; the API exchanges compact JSON
Block I/O	126 MB / 12.3 kB	1.25 GB / 4.1 kB	The API read significantly more data because of the libraries and model weights
PIDs	11	29	The ML runtime and API server use more processes/threads

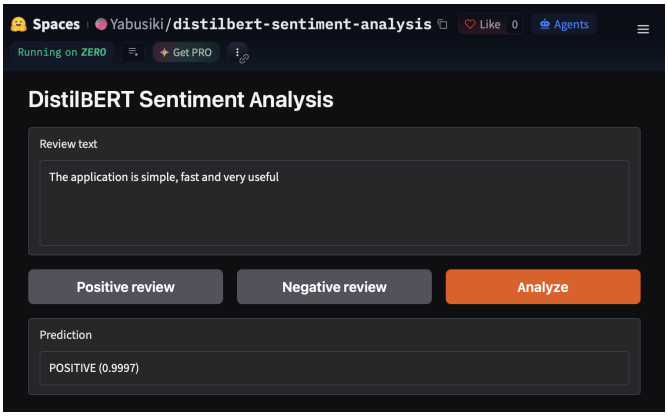


Figure 7: Hugging Face Space run.

6.6. Online Inference

Two ways to call the Space were tested: through the web interface (Figure 7) and through the API (Figure 8).

The `scripts/call_hf_space.py` script uses the Gradio Client:

```
HF_SPACE = os.environ["HF_SPACE"]

def main() -> None:
    client = Client(HF_SPACE)
    result = client.predict(
        "I really enjoyed this assignment.",
        api_name="/classify_text",
    )
    print(result)
```

The Space name is passed through an environment variable. The client connects to the published Space and calls the endpoint.

```
docker run --rm \
  -e HF_SPACE=Yabusiki/distilbert-sentiment-analysis \
  --mount "type=bind,source=$PWD/scripts,target=/scripts,readonly" \
  distilbert-ui:latest \
  python /scripts/call_hf_space.py
```

The parameters mean:

- `--rm` — remove the temporary container after the call;
- `-e HF_SPACE=...` — pass the Space name;
- `type=bind` — mount an existing Mac directory;
- `source=$PWD/scripts` — the source on the host;
- `target=/scripts` — the path inside the container;
- `readonly` — prevent the container from modifying the local scripts;
- `distilbert-ui:latest` — use the lightweight image in which Gradio Client is already installed as part of Gradio;
- the last line starts the API client.

```
toma@Tamaras-MacBook-Air docker-dl-assignment % docker run --rm \
  -e HF_SPACE=Yabusiki/distilbert-sentiment-analysis \
  --mount "type=bind,source=$PWD/scripts,target=/scripts,readonly" \
  distilbert-ui:latest \
  python /scripts/call_hf_space.py
Loaded as API: https://yabusiki-distilbert-sentiment-analysis.hf.space
POSITIVE (0.9999)
toma@Tamaras-MacBook-Air docker-dl-assignment %
```

Figure 8: A temporary local Docker container connected to the public Space.

7. Conclusion

As a result, a sentiment analysis application based on pre-trained DistilBERT was created and verified. In the main version, ML inference and the graphical interface are isolated in separate Docker containers and connected through a local FastAPI using Docker Compose. In addition, the application was published in Hugging Face Spaces on ZeroGPU.

The Dockerfiles are organized with layer caching in mind, the model is saved in the API image during the build, and Compose defines the health dependency, RAM limits, and shared memory. The `docker stats` analysis confirmed the expected separation of resources: the inference container uses more memory, block I/O, and processes, while the UI container remains relatively lightweight.

The project demonstrated the main principles of deploying ML systems: a reproducible environment, separation of service responsibilities, network communication, readiness control, resource management, and moving an application to a cloud environment.